

VPGFUZZ: Vulnerable Path-Guided Greybox Fuzzing

Zhechao Lin^{ID}, Jiahao Cao^{ID}, *Member, IEEE*, Xinda Wang^{ID}, Renjie Xie, Yuxi Zhu^{ID}, Xiao Li, Qi Li^{ID}, *Senior Member, IEEE*, Yangyang Wang, and Mingwei Xu^{ID}, *Senior Member, IEEE*

Abstract—Fuzzing is a prevalent technology for identifying software vulnerabilities. Existing fuzzing techniques predominantly focus on maximizing code coverage to unearth potential security issues. However, the mere expansion of explored code does not necessarily correlate with an increased discovery of vulnerabilities. Additionally, existing fuzzers often neglect comprehensive execution path information in code exploration. Consequently, potential vulnerabilities may be delayed or overlooked in the fuzzing process. To address this, we propose VPGFUZZ, a vulnerable path-guided fuzzer that can not only explore new code but also exploit known vulnerability path knowledge for vulnerability discovery. It employs a vulnerable path recognition model to identify test cases with potentially vulnerable paths. This model is trained with various execution paths derived from real-world vulnerability PoCs (Proof of Concepts). Based on this model, VPGFUZZ applies an explore-exploit seed selection strategy to effectively choose test cases for testing. Unlike traditional seed selection methods that maintain a single queue for exploring new code, this strategy includes a separate queue for retaining test cases identified as potentially vulnerable, allowing for more thorough testing. Experimental results demonstrate that VPGFUZZ discovers 24 previously unknown vulnerabilities, with 18 receiving vulnerability identifiers from third-party organizations such as CVE. Our evaluation also shows VPGFUZZ’s superior efficiency by uncovering the first vulnerability approximately 1.2 to 70 times faster than popular fuzzers in most programs.

Index Terms—Greybox fuzzing, vulnerable path, recognition model, explore-exploit seed selection.

I. INTRODUCTION

FUZZING is a widely recognized technique for detecting software vulnerabilities. By inputting a large amount of

random and unexpected test cases into the target program, this technology can monitor the program behaviors to identify potential security vulnerabilities. With the ease of deployment, remarkable scalability, and broad applicability, fuzzing has become a staple in industry practices. Notably, the Google OSS-Fuzz [1] platform integrates various fuzzers [2], [3], [4], [5], contributing to the discovery of 10,000 vulnerabilities across 1,000 open-source projects.

In the fuzzing process, while the random use of extensive test cases can theoretically expose a wide range of vulnerabilities, it often results in significant overhead, as some cases often trigger redundant similar program behaviors without revealing vulnerabilities. To address this, coverage-guided fuzzing [2], [6], [7], [8], [9], [10], [11] emerges as a solution by leveraging code coverage metrics to steer the exploration of program behaviors more effectively. A notable example of this technology is AFL [2], which transforms the fuzzing process into an optimization problem: maximizing explored code for a given program. The underlying insight is straightforward: expanding the scope of explored code increases the probability of triggering vulnerabilities.

While coverage-guided fuzzing has proven efficacy in various scenarios, building a fuzzer that is both effective and efficient for large-scale programs presents significant challenges. These challenges are primarily due to the constraints of limited resources and the pressing need to discover as many vulnerabilities as possible in a constrained timeframe.

Challenge #1. To effectively discover unknown vulnerabilities, fuzzing should trigger as many execution paths as possible. However, real-world programs inherently contain an extensive number of paths. Consequently, it is impractical to rely on path coverage as the guidance for exploring potential vulnerabilities. Alternatively, coverage-guided fuzzers often use edge coverage to guide their exploration, namely, only focusing on test cases that trigger previously unexplored edges. This strategy, while efficient, lacks sensitivity to those paths that may not introduce new edges but still reveal different code information. Consequently, the identification of vulnerabilities in these paths may be overlooked or delayed.

Challenge #2. Employing various code coverage metrics, current coverage-guided fuzzers can comprehensively explore programs to uncover vulnerabilities. To enhance the efficiency of vulnerability discovery, recent studies have proposed several seed selection methods aimed at prioritizing test cases with a higher propensity for revealing vulnerabilities, such as AFLFast [12], CollaAFL [13], and TortoiseFuzz [14]. These methods primarily rely on intuitive heuristics to guide their selection process. However, such heuristics may not accurately

Received 22 July 2024; revised 29 June 2025; accepted 12 August 2025. Date of publication 8 September 2025; date of current version 16 September 2025. The work of Zhechao Lin, Jiahao Cao, Renjie Xie, Yuxi Zhu, Xiao Li, Qi Li, Yangyang Wang, and Mingwei Xu was supported by the National Natural Science Foundation of China (NSFC) under Grant 62202260, Grant 62221003, Grant 62472246, Grant 62132011, and Grant 623B2062. The associate editor coordinating the review of this article and approving it for publication was Dr. Yan Wang. (Corresponding authors: Jiahao Cao; Mingwei Xu.)

Zhechao Lin, Yuxi Zhu, and Xiao Li are with the Department of Computer Science and Technology, Tsinghua University, Beijing 100084, China (e-mail: zclin21@gmail.com; zyx23@mails.tsinghua.edu.cn; li-x23@mails.tsinghua.edu.cn).

Jiahao Cao, Renjie Xie, Qi Li, and Yangyang Wang are with the Institute for Network Sciences and Cyberspace, Tsinghua University, Beijing 100084, China (e-mail: caojh2021@tsinghua.edu.cn; xrj21@mails.tsinghua.edu.cn; qli01@tsinghua.edu.cn; wangyy-13@tsinghua.edu.cn).

Xinda Wang is with the Department of Computer Science, The University of Texas at Dallas, Richardson, TX 75080 USA (e-mail: xinda.wang@utdallas.edu).

Mingwei Xu is with the Department of Computer Science and Technology and the Institute for Network Sciences and Cyberspace, Tsinghua University, Beijing 100084, China (e-mail: xmw@cernet.edu.cn).

Digital Object Identifier 10.1109/TIFS.2025.3607249

1556-6021 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission.

See <https://www.ieee.org/publications/rights/index.html> for more information.

Authorized licensed use limited to: Harbin Engineering Univ Library. Downloaded on August 23, 2026 at 14:28:11 UTC from IEEE Xplore. Restrictions apply.

identify the complex patterns that characterize vulnerabilities in execution paths. As a result, some test cases critical for triggering vulnerabilities may be overlooked. For instance, AFLFast focuses on test cases that explore paths of lower frequency; however, the rarity of a path does not inherently signify a vulnerability. Additionally, while CollAFL and TortoiseFuzz consider security-sensitive attributes for edges in their selection logic, the presence of these security-sensitive edges may not reveal vulnerabilities. Particularly, some memory protection mechanisms often involve numerous sensitive memory operations.

Our Solution. To address the challenges, we propose VPGFUZZ, a vulnerable path-guided fuzzer that can effectively and efficiently unveil vulnerabilities in large-scale programs. VPGFUZZ comprises two key components: *vulnerable path recognition* and *vulnerable path-guided fuzzing*. The vulnerable path recognition can help the fuzzing component prioritize test cases that likely trigger vulnerable execution paths. Based on a machine learning model, our vulnerable path recognition is designed to automatically identify execution paths that contain potential vulnerabilities. To train this model, we collect a dataset of execution paths from publicly available datasets. Specifically, we retrieve 3138 PoCs of vulnerabilities from CVE [15], OSS-Fuzz, and GitHub, and then run them to obtain corresponding path information. To achieve an accurate vulnerable path recognition, we extract vulnerability-relevant structural and statistical features from path information.

Unlike existing coverage-guided fuzzers, our vulnerable path-guided fuzzing is designed to not only explore new paths for discovering unknown vulnerabilities but also to exploit those that are identified as potentially vulnerable. Based on the information from vulnerable path recognition, this fuzzing component uses an *explore-exploit* seed selection strategy, optimizing the selection of seeds (i.e., test cases) for subsequent testing. This strategy uses the *exploit* and *explore* queue to prioritize seeds. Specifically, seeds identified as likely to trigger vulnerabilities are allocated to the *exploit* queue, while those that contribute to new code coverage are assigned to the *explore* queue. Notably, we first select seeds from the *exploit* queue, ensuring that potential vulnerabilities are addressed promptly. When all seeds are tested and no new vulnerable seeds are found, we switch to the *explore* queue. While prior studies like EcoFuzz [16], AFL-Hier [17], and Entropic [18] propose strategies to balance exploration and exploitation—using multi-armed bandits or entropy—they remain within the coverage-guided paradigm, driven by code-coverage metrics. In contrast, our approach prioritizes seeds by their predicted vulnerability-triggering potential, balancing fast vulnerability exposure with broader coverage and mitigating premature convergence.

We evaluate VPGFUZZ on 12 widely-used real-world programs, comparing it against four popular fuzzers [2], [12], [13], [14]. Our results indicate that VPGFUZZ identifies 24 previously unknown vulnerabilities. Of these, 18 receive vulnerability identifiers from third-party organizations such as CVE, and 6 are assessed as high-severity. Additionally, VPGFUZZ demonstrates superior efficiency in discovering vulnerabilities across most programs, consistently achieving

the fastest Time to First Vulnerability (TFV) in the majority of cases. For specific programs, such as MP3gain, VPGFUZZ exposes vulnerabilities more than 70 times faster than previous studies. Despite a 6% decrease in the average unique paths explored, VPGFUZZ identifies 23% more unique crashes and 14% more vulnerabilities than the best results of other fuzzers. Our evaluation also demonstrate an acceptable overhead introduced by VPGFUZZ. On average, its fuzzing throughput is 10.69% lower than AFL's and 1.72% higher than TortoiseFuzz's.

In summary, this paper makes the following contributions:

- We design VPGFUZZ, a vulnerable path-guided fuzzer. Unlike existing coverage-guided fuzzers, it identifies and exploits test cases that are likely to trigger vulnerabilities.
- We develop a vulnerable path recognition model, which can help our fuzzer to identify test cases that potentially trigger vulnerable paths.
- We propose a seed selection strategy based on our vulnerable path recognition model, enabling our fuzzer to trigger vulnerabilities at a faster rate.
- We construct a dataset containing 3138 vulnerability PoCs, including PoC files and corresponding program versions, and the commands triggering the vulnerabilities. We will publish it later.
- We implement the prototype of VPGFUZZ and conduct experiments to validate its effectiveness and efficiency in vulnerability discovery compared to existing methods.

II. SYSTEM DESIGN

In this section, we present a motivating example that illustrates how existing coverage-guided fuzzing techniques may miss potential vulnerabilities due to a lack of path information. Following this, we introduce VPGFUZZ, a vulnerable path-guided fuzzer designed to effectively identify vulnerabilities by leveraging diverse path characteristics.

A. Motivating Example

Figure 1 illustrates a motivating example, which demonstrates the possible problem in coverage-guided fuzzing. Figure 1(a) is a C-language code snippet that contains a Use-After-Free (UAF) vulnerability, and Figure 1(b) shows the corresponding control flow graph. The arrows of different colors represent the execution paths of different test cases. To trigger this vulnerability, two conditions need to be satisfied: the execution path should first deallocate the memory pointer $p1$ in line 8, and then use the content of $p1$ in line 17.

Assume there are three test cases, specifically $case_0$ ('ma'), $case_1$ ('ae'), and $case_2$ ('me'). As Figure 1(b) shows, the execution path of $case_0$ passes through the free point in line 8, but in the next program branch, the constraint condition of entering the use point is not satisfied. Therefore, it does not satisfy the conditions to trigger the vulnerability. Additionally, while the execution path of $case_1$ satisfies the constraint condition to enter the use point, it cannot execute the free instruction in line 8 to trigger the vulnerability. Conversely, running the program with $case_2$ can pass through the free point in line 8 and also enter the path containing the use point in line

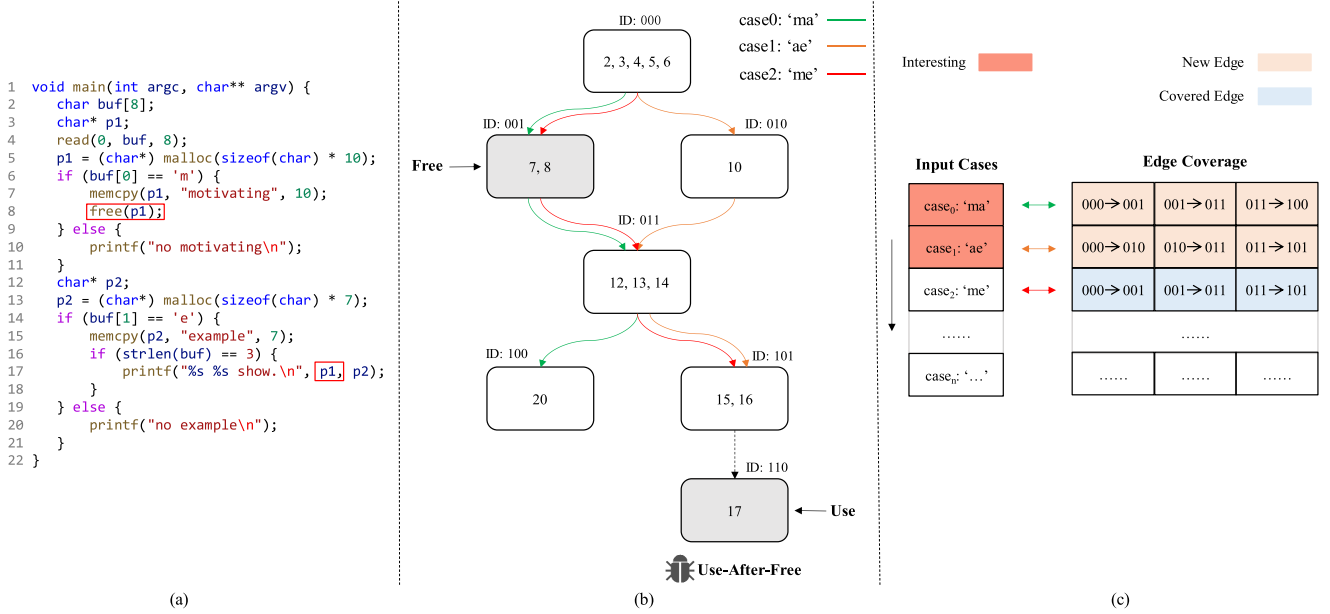


Fig. 1. A motivating example on missing potential vulnerabilities without path information: (a) source code; (b) control flow graph; (c) edge coverage for different seeds.

17. This means, compared with $case_0$ and $case_1$, the test case $case_2$ is more likely to trigger the vulnerability intuitively.

Figure 1(c) presents the status of edge coverage when the program runs with $case_0$, $case_1$, and $case_2$, respectively. Notably, while $case_2$ is more likely to trigger the vulnerability, it may not be prioritized in the seed queue for further fuzzing. This is because the edges activated by $case_2$ are not new; they have been covered by $case_0$ and $case_1$. For instance, $case_0$ covers the edges $000 \rightarrow 001$ and $001 \rightarrow 011$, and $case_1$ covers the edge $011 \rightarrow 101$. Consequently, the test case $case_2$, failing to trigger new edges, will not be recognized as an interesting seed for subsequent fuzzing. Thus, the edge coverage-guided fuzzing may inadvertently overlook or delay the discovery of specific vulnerabilities.

1) Key Insight: As the motivating example shows, coverage-guided fuzzing often considers simple edge code coverage and does not actively exploit execution paths that likely contain vulnerability patterns. However, it is clear that vulnerable paths exhibit distinctive characteristics compared to non-vulnerable paths for specific vulnerabilities, e.g., some objects are used again after being freed in the path of a UAF vulnerability. Therefore, we can utilize the characteristics of various vulnerability paths to identify test cases that may trigger vulnerabilities and prioritize them in the seed queue, even if they do not activate new edges.

B. VPGFuzz Overview

As we aforementioned in the motivating example (§ II-A), existing coverage-guided fuzzers, such as AFL [2] and AFLFast [12], often use path-insensitive code coverage criteria to discover vulnerabilities. Additionally, they do not introduce an approach to prioritize test cases that likely trigger vulnerabilities. As a result, they may miss or delay the discovery of potential vulnerabilities.

To address this, we propose VPGFUZZ, a vulnerable path-guided fuzzer that can effectively discover program codes related to vulnerabilities. As depicted in Figure 2, VPGFUZZ's pipeline consists of two core components: (i) *vulnerable path recognition*, and (ii) *vulnerable path-guided fuzzing*. Overall, VPGFUZZ operates the two components in the *pre-fuzzing* and *fuzzing* phase. In the pre-fuzzing phase, VPGFUZZ collects various execution paths and uses them to train a model for (i), aiming to learn the characteristics of potentially vulnerable paths. In the later fuzzing phase, VPGFUZZ first runs the program with test cases, recognizes potentially vulnerable paths using (i), and then prioritizes the corresponding test case for the subsequent fuzzing process in (ii).

1) Vulnerable Path Recognition: To identify paths that are more likely to contain vulnerabilities, we propose a vulnerable path recognition model to learn the code characteristics of potentially vulnerable paths. Specifically, the recognition model undergoes training with a variety of execution paths collected from real-world programs in CVE and OSS-Fuzz projects. Notably, a path is labeled 'vulnerable' if exist vulnerability proofs of concept (PoCs) that can trigger the path. If a path is yet not found to contain any vulnerabilities, it is considered 'secure' (or non-vulnerable). After training, the model can help our VPGFUZZ recognize the possibility of an execution path containing a vulnerability. If the possibility exceeds a certain threshold, the corresponding test case is preserved as an interesting seed in the queue for subsequent fuzzing. More details can be found in § III.

2) Vulnerable Path-Guided Fuzzing: Distinguished from conventional coverage-guided fuzzers, our VPGFUZZ extends its exploration capabilities beyond merely uncovering new edges. It distinctly prioritizes seeds (*i.e.*, test cases) with the potential to activate vulnerable paths. As depicted in Figure 2, this is achieved by adopting two queues to schedule fuzzing

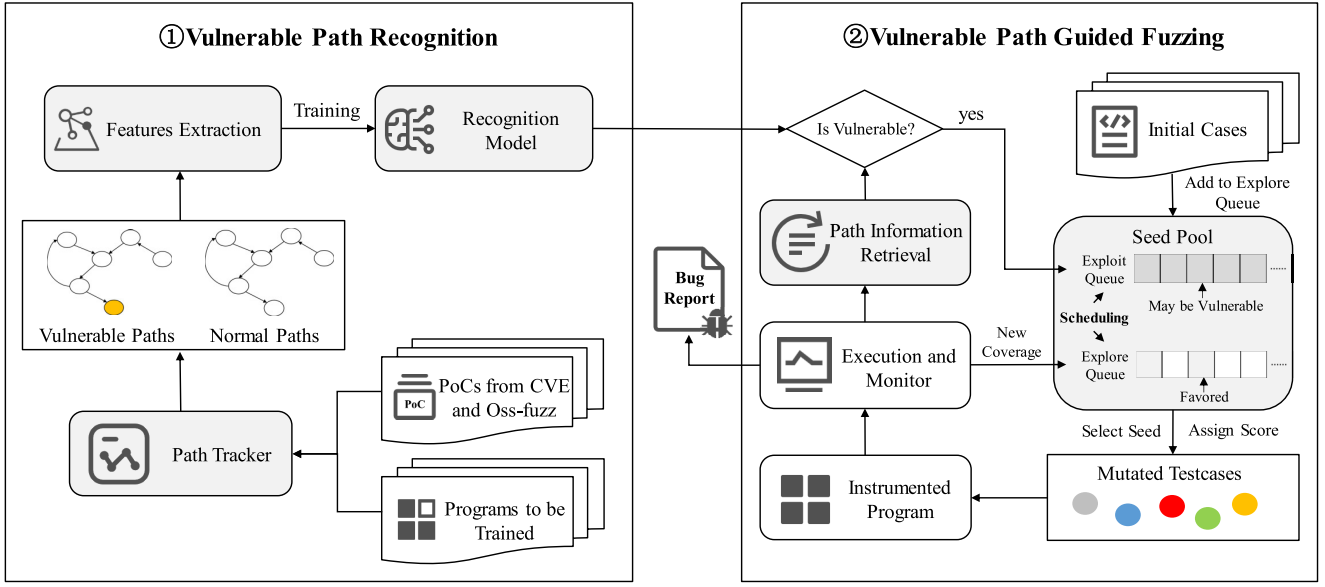


Fig. 2. VPGFUZZ overview.

seeds, i.e., *explore queue* and *exploit queue*. To be precise, seeds in the *explore queue* are evaluated by the original fitness function, while the seeds in the *exploit queue* are evaluated by a tailored fitness function. This tailored fitness function utilizes our vulnerable path recognition model to assess the possibility of seeds triggering vulnerable paths and accordingly assign priority scores to the seeds. The seeds in the *explore queue* possess the potential to trigger vulnerabilities. VPGFUZZ prioritizes selecting seeds from the *exploit queue*. Once all seeds in the *exploit queue* have been tested without discovering new ones, VPGFUZZ shifts focus to the *explore queue*. With the integration of the two queues, VPGFUZZ not only broadens its exploration of code snippets but also enhances its effectiveness in discovering vulnerabilities by focusing on test cases with a higher possibility of triggering critical vulnerabilities. More details are present in § IV.

III. VULNERABLE PATH RECOGNITION

This section introduces our vulnerable path recognition module, which aims to identify paths that are more likely to contain vulnerabilities. It comprises three components: *path tracker*, *feature extraction*, and *recognition model*.

A. Path Tracker

To capture path information during program execution, we develop a path-tracking tool designed to monitor the sequence of instructions activated by a test case. Although traditional dynamic instrumentation tools offer a basic framework for path tracking, they have limitations and are not suitable for our path-tracking task. For example, while NeuFuzz [19] leverages Intel PT technology to facilitate efficient path tracking, this approach lacks scalability due to its reliance on specific hardware and operating system support. The Callgrind [20] tool can be utilized for efficient path tracking [21], but it focuses

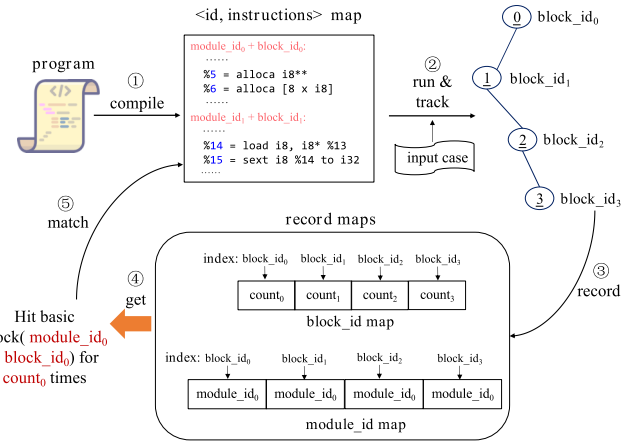


Fig. 3. Workflow of path tracking.

solely on tracing function call sequences without capturing low-level instruction details that are essential for a thorough analysis of vulnerability features (Section III-B).

To mitigate performance overhead without reliance on specialized software and hardware, we propose a new pack-tracking method based on instrumentation. As depicted in Figure 3, this method assigns a unique hash ID, ranging from 0 to $2^{16} - 1$, to each basic block in the compilation process. Following AFL, the ID serves as the index to log the execution frequency of the block into an array. In practice, we observe hash conflicts may typically occur within the ID range of $[0, 2^{16} - 1]$. Consequently, multiple different basic blocks may be allocated with the same block ID.

One naive method for mitigating this conflict risk is to expand the value range of block IDs; however, this solution significantly increases computation overhead, rendering it impractical for path tracking. To address this challenge, we assign a unique module ID to each module of the program, ranging from 0 to $2^8 - 1$. With the integration of module

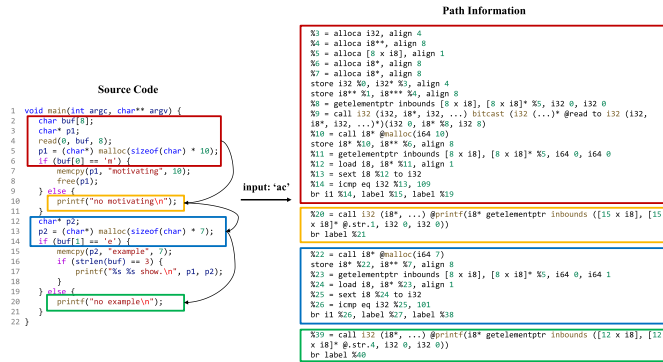


Fig. 4. Example of full path information.

IDs, each basic block can be distinctly identified by the combination of the corresponding module ID and its block ID. This methodology effectively minimizes the likelihood of hash collisions while maintaining a reasonable overhead. The efficiency of this approach is further validated in our experiments.

As shown in Figure 3, we use two key-value maps to track path execution information, namely, the `block_id` map and the `module_id` map. In the `block_id` map, the element associated with key B records the execution frequency of basic block B . Concurrently, the `module_id` map uses the element corresponding to key B to log the ID of the module that contains basic block B . When running the program with a test case, this dual-mapping strategy provides the following information.

module_id + *block_id* : *count_i* p>Here, *count_i* represents the execution time of basic block *module_id* + *block_id*. In addition to the path execution information, we also utilize a key-value map, denoted as $\langle id, instructions \rangle$, to store associated block instructions for further analysis.

B. Path Feature Extraction

In real-world programs, full path information often includes code unrelated to vulnerabilities. Figure 4 presents an example generated using a path tracking tool developed by us. The left sub-figure shows a partial code snippet; when the input is ‘ac’, the corresponding path information is shown on the right. Colored boxes highlight source code and LLVM IR instructions within different basic blocks. As illustrated, many instructions—comprising various opcodes and operands—are irrelevant to vulnerability triggering. This underscores the need for distinguishing features to identify potentially vulnerable paths.

We aim to identify features within paths that are relevant to vulnerabilities, including both structural and statistical characteristics. As shown in Table I, based on extensive analysis and our experience with numerous CVE-reported vulnerabilities, we have identified the path length, the maximum number of loops within a path, and the number of modules spanned by the path as structural features. Typically, longer program execution paths suggest increased code complexity, which in turn heightens the risk of vulnerabilities. Excessive iterations within loops can lead to resource exhaustion issues, while

TABLE I
STRUCTURAL AND STATISTICAL PATH FEATURES

Type	Feature Name
Structural Path Features	Path length
	Max No. of loops within a path
	No. of modules span by path
Statistical Path Features	No. of memory operations
	No. of pointer operations
	No. of arithmetic operations
	No. of conditional operations

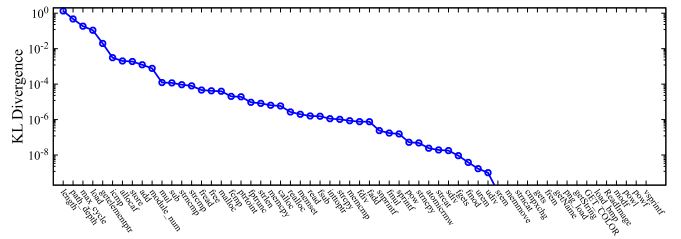


Fig. 5. KLD of statistical features.

TABLE II
SELECTED FUNCTIONS AND INSTRUCTIONS THAT ARE USED TO DERIVE
STATISTICAL FEATURES

Operations	Functions	Instructions
Memory	allocate	malloc, realloc, calloc alloca
	free	free
	read	memcpy, strcmp, strncmp, strlen load
	write	memset, read, fread, fgets store, atomicrmw
	copy	memcpy, strcpy, strncpy
	format	strcat, sprintf, snprintf
Pointer		getelementptr, fptrunc, ptrtoint, inttoptr
Arithmetic	pow, fmod	add, fadd, sub, fsub, mul, fmul, udiv, sdiv, fdiv, urem
Conditional	memcmp, strcmp, strncmp	icmp, fcmp

improper loop control might cause out-of-bounds access, a frequent source of vulnerabilities. Additionally, interactions and interface calls across different modules can also increase the potential for vulnerabilities. KARONTE [22] highlights this risk by proposing a method for detecting vulnerabilities introduced by unsafe interactions between modules.

For statistical features, we focus on the count of dangerous operations, such as specific memory, pointer, and arithmetic operations. This is because a higher occurrence of these dangerous operations along a path increases the likelihood of triggering vulnerabilities. Additionally, we note that security-aware developers tend to enforce constraints on dangerous operations. Consequently, operations with fewer constraints are more likely to trigger vulnerabilities. Therefore, we add the number of conditional operations, such as those constrained based on comparison results, into our statistical feature set.

To further identify more relevant statistical features for vulnerability path recognition, we also investigate CVE-reported vulnerabilities in recent years and functions/instructions that are frequently involved in these vulnerabilities.

Based on our investigation, we carefully extract several function-level and instruction-level statistical features. To

TABLE III
PROGRAMS FOR COLLECTING PATH INFORMATION

Type	Project	Program
Text	cflow	cflow
	jq	jq
	mujs	mujs
	libxml2	xmllint
	nasm	nasm
Image	exiv2	exiv2
	jhead	jhead
		tiffsplit, tiff2bw, tiffset, tiff2pdf,
	libtiff	tiff2ps, tiff2rgba, tiffcp, tiffcrop,
		tiffmedian, tiffinfo, pal2rgb
Audio	gdk-pixbuf	gdk-pixbuf-pixdata
	jasper	jasper, imginfo
	lame	lame
	mp3gain	mp3gain
	swftools	wav2swf
Video		makeswf, swftocxx, swftophp,
	libming	swftopython, listmp3,
		listswf, dbl2png
	flvmeta	flvmeta
	Bento4	acc2mp4, mp42aac, mp4dump,
Binary		mp42hls, mp4audioclip, mp4tag
	ffmpeg	ffmpeg, ffmpeg_g
		addr2line, ld-new, objdump,
	binutils	objcopy, readelf, strip-new,
		nm-new, size
Network	tcpdump	tcpdump
	tcpreplay	tcpdump, tcpwrite
Total PoCs: 3138		

evaluate the effectiveness of these features in distinguishing between vulnerable and non-vulnerable paths, we compute the Kullback–Leibler Divergence (KLD) between the distributions of structural and statistical features on each path type. The results show that most features can serve as useful indicators for identifying vulnerable paths. As illustrated in Figure 5, we sorted the features in descending order of their KLD values. Some features exhibited a KLD value of 0, indicating their inability to differentiate between the two path types. Consequently, we exclude these features. Our final selected functions and instructions that are used to derive statistical features are shown in Table II.

C. Vulnerable Path Recognition Model

1) *Path Collection*: To train our vulnerable path recognition model, we collect a dataset comprising various vulnerable and non-vulnerable paths. Initially, as shown in Table III, we select 20 popular real-world projects [23], encompassing the six most common software types: *Text*, *Image*, *Audio*, *Video*, *Binary*, and *Network*. We then employ our path tracking tool to collect vulnerable path information from the corresponding publicly available PoCs of vulnerabilities. Specifically, these PoCs are sourced from References in CVEs [15] as well as reproducer

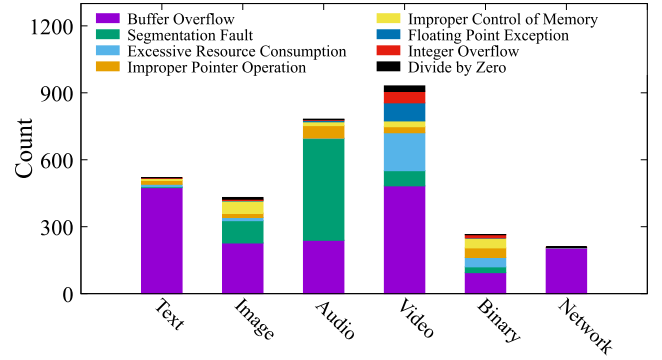


Fig. 6. Distribution of PoCs for different vulnerability types in common software types.

test case, minimized test case, and unminimized test case in OSS-Fuzz [1], and issues in GitHub.

We gather execution paths from 3138 PoCs that span a range of common vulnerability types, such as buffer overflow, improper pointer operations (including double free, use after free, NULL pointer dereference.), improper control of memory (e.g., memory leak, out of bounds read/write), integer overflow and divide by zero issues. The distribution of PoCs for different vulnerability types is depicted in Figure 6. It's worth noting that each collected execution path corresponds to a unique PoC.

It is necessary to collect a dataset of non-vulnerable paths as ground truth. Here, like previous work [24], we assume that code without known vulnerabilities is non-vulnerable. From the 20 software applications we collect, we randomly select 3138 paths that have not yet encountered vulnerabilities and then label them as non-vulnerable.

2) *Model Training*: With diverse non-vulnerable and vulnerable paths, we can train a vulnerable path recognition model that can assess the possibility of a given execution path triggering a vulnerability. Notably, vulnerability paths contain a multitude of features that span various dimensions, with each feature making a distinct contribution to the determination of vulnerability paths. Consequently, intricate patterns and relationships may be concealed within these features. Therefore, it is necessary to leverage a machine learning model for vulnerable path recognition. Since designing a machine learning model is not the core contribution of our work and various models are readily available, we will conduct a comparative study in Section V-A across six popular learning algorithms: Random Forest and Extra Trees (ensemble tree models), LightGBM and XGBoost (gradient-boosting frameworks), Multi-Layer Perceptron (neural networks), and Logistic Regression. The comparison focuses on their ability to recognize vulnerable paths. It is worth noting that our framework is model-agnostic and can accommodate different machine learning models.

IV. VULNERABLE PATH-GUIDED FUZZING

With the integration with our vulnerability path recognition model, our VPGFUZZ is capable of assessing the likelihood of test cases' execution paths triggering vulnerabilities. The

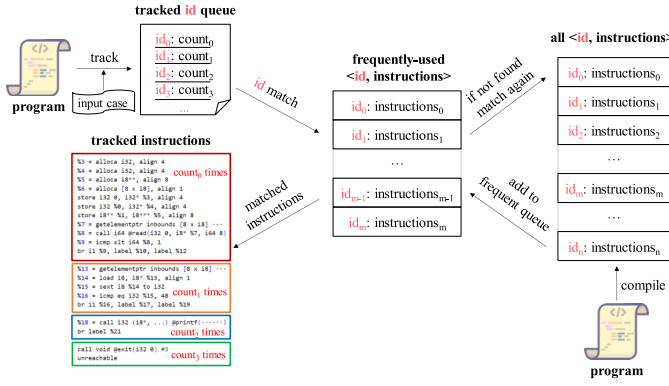


Fig. 7. Workflow of path information retrieval.

possibility information then guides VPGFUZZ's subsequent fuzzing strategy. As depicted in Figure 2, VPGFUZZ first retrieves path information and then estimates the probability of triggering vulnerabilities. When this probability surpasses a predefined threshold, the test case is classified as having a high probability of triggering vulnerabilities and is accordingly enqueued into an `exploit_queue`. Leveraging this queue, we develop specialized fuzzing components, including *path information retrieval*, *enhanced test case filter*, *explore-exploit seed selection*, and *mixed score assignment*.

A. Path Information Retrieval

Intuitively, we can use our path-tracking tool to collect path information during program execution, including the set of instructions executed in basic blocks and the execution frequency of basic blocks. However, this method is impractical when the tested program is large. The path information not only needs to be retrieved for training the vulnerable path recognition model but also needs to be retrieved online for the fuzzing process. When the number of basic blocks becomes large, it will be time-consuming to collect path information and record executed instructions.

To address this challenge, we follow the design of the second-level cache [25] to propose a basic block cache mechanism. Given the fact that only a part of all the basic blocks may be executed during program execution, we allocate a space in the memory, denoted as the `< id, instructions >` map, to cache the basic blocks most commonly executed. As depicted in Figure 7, we continuously search executed basic blocks from the cache map. If a basic block is not found, we then retrieve it from the original maps generated in the compilation phase and then add it to the cache map.

B. Enhanced Test Case Filter

Off-the-shelf coverage-guided fuzzers, such as AFL, conceptualize the fuzzing process as genetic evolution, and they follow the principles of traditional genetic algorithms to explore potential program behaviors. To steer the exploration efforts towards discovering more code snippets, they leverage edge coverage between basic blocks as a pivotal metric within the fitness function. The fitness function is utilized to identify 'interesting' test cases in the seed pool, facilitating further

Algorithm 1 Enhanced Test Case Filtering Algorithm

Input: `bb_inst_map` recording all block_id and its instructions, test case `i`, `block_id_map_i` recording the count of basic block hits for `i`, `cur_bitmap_i` recording edge coverage for `i`, `all_bitmap` recording edge coverage for all test cases

Output: if `i` is interesting return 1, else return 0

```

1: function SAVE_IF_INTERESTING(i)
2:   result ← 1
3:   LoadData(bb_inst_map)
4:   path_info ← RetrievalPath(bb_inst_map, block_id_map_i)
5:   vul_prob ← RecognitionModel(path_info)
6:   if vul_prob ≥ THRESHOLD then
7:     AddCasetoQueue(i)
8:     return result
9:   end if
10:  if HasNewEdge(cur_bitmap_i, all_bitmap) ||
    HasNewHitCount(cur_bitmap_i, all_bitmap) then
11:    AddCasetoQueue(i)
12:  else
13:    result ← 0
14:  end if
15:  return result
16: end function

```

mutation and fuzzing. A test case is deemed interesting if it either activates previously uncovered edges or triggers known edges at a new scale, as demonstrated below:

$$IsInteresting(s) = \begin{cases} true, & NewTuple(s) \\ & \vee NewHitCount(s) \\ false, & otherwise \end{cases} \quad (1)$$

As we aforementioned in the motivating example (§ II-A), existing coverage-guided fuzzers primarily use edge coverage in their fitness functions to prioritize test cases, often overlooking the comprehensive execution path information and relevant vulnerability characteristics. As a result, certain test cases that do not introduce new edges but activate new vulnerable paths may be unjustly overlooked, tagged as uninteresting, and discarded. Such oversight potentially delays or misses the identification of potential vulnerabilities. To mitigate this issue, we leverage the retrieved path information and employ the recognition model to estimate the probability of a path containing vulnerabilities. When the probability surpasses a predefined threshold, irrespective of whether the test case triggers new edges, we incorporate it in the seed pool for subsequent testing. This process is shown in Algorithm 1.

C. Explore-Exploit Seed Selection

Our VPGFUZZ maintains two queues to prioritize test cases in the seed queue for efficient fuzzing, namely, `explore_queue` and `exploit_queue`. Following AFL, `explore_queue` is designed as a higher-priority queue that preserves test cases that hit new edges. The queue contains the minimum subset of test cases covering all currently hit edges. For each edge, `explore_queue` records the best candidate that covers the edge, denoted as *favor*. Once starting a new fuzzing loop, VPGFUZZ first prioritizes seeds in `explore_queue` and then selects those labeled as *favor* for subsequent execution.

A test case `case_i` is labeled as *favor* and subsequently enqueued into the `explore_queue` if it satisfies at least one

of the following conditions: (i) it activates an edge, $edge_j$, that has not been covered by any prior test case or (ii) it demonstrates the lowest product of execution time and file size in comparison to previous ones, indicating optimal runtime efficiency. The latter condition highlights a seed selection strategy that prioritizes seeds offering minimal execution overhead, thereby enhancing overall testing efficiency and throughput. The formal equation is presented as follows:

$$explores_queue[edge_j] = \begin{cases} case_i, & (cur_bitmap_i[edge_j] > 0 \\ & \wedge explores_queue[edge_j] = \emptyset) \\ & \vee IsMin(exec_time_i * size_i) \\ \emptyset, & otherwise \end{cases} \quad (2)$$

To exploit test cases that potentially trigger vulnerabilities, VPGFUZZ utilize *exploit_queue* to preserve test cases that likely trigger vulnerable paths. *exploit_queue* relies on the vulnerable path recognition model to preserve and prioritize test cases. Specifically, it uses the model to assess the probability of every test case triggering vulnerabilities based on its path information. When this probability exceeds a threshold, the test case is considered dangerous and then enqueued into *exploit_queue*. Notably, all test cases in *exploit_queue* are ranked by the probability.

With equipment of *exploit_queue*, VPGFUZZ can effectively discover test cases that potentially trigger vulnerabilities. It's essential to note that VPGFUZZ also maintains *explores_queue* to store test cases that cover new edges, which is similar to AFL's fuzzing. The main principle is that while we focus on the dangerous test cases likely triggering vulnerabilities, we should also ensure VPGFUZZ to explore more program behaviors, aiming to find unknown vulnerabilities.

Algorithm 2 Explore-Exploit Seed Selection Algorithm

Input: Original seed queue *explores_queue*, Exploitable seed queue *exploit_queue*

Output: seed for further mutation

```

1: function NEXTSEED(explores_queue, exploit_queue)
2:   seed ← null
3:   if IsNotEmpty(exploit_queue) then
4:     for i=0 → Length(exploit_queue) do
5:       if exploit_queue[i].fuzzed == 0 then
6:         seed ← exploit_queue[i]
7:         exploit_queue[i].fuzzed ← 1
8:       end if
9:     end for
10:  else
11:    for j=0 → Length(explores_queue) do
12:      if explores_queue[j].favor == 1 &&
explores_queue[j].fuzzed == 0 then
13:        seed ← explores_queue[j]
14:        explores_queue[j].fuzzed ← 1
15:      end if
16:    end for
17:  end if
18:  return seed
19: end function

```

Algorithm 2 shows the seed selection process. VPGFUZZ gives priority to mutating seeds from the exploit queue, that is,

VPGFUZZ prioritizes seeds that are more likely to trigger vulnerabilities. When all dangerous test cases in *exploit_queue* are executed, and no ones are discovered, VPGFUZZ switches back to the original AFL mode and prioritizes *favor* seeds from *explores_queue*, aiming to explore more execution paths and discover their potential vulnerabilities.

D. Mixed Score Assignment

Based on our vulnerable path recognition model, we can obtain the probability of test cases exhibiting vulnerable execution paths. By using this probability as the priority score for the *exploit_queue*, VPGFUZZ can easily select test cases that are more likely to trigger vulnerabilities and then incorporate them in the subsequent mutation process.

Before mutation, VPGFUZZ should assign mutation scores to test cases in *explores_queue* and *exploit_queue*, indicating their mutation number. For a test case in *explores_queue*, its mutation score $score_{ori}$ is proportional to four aspects: execution time, covered edges, number of test rounds, and path depth. The calculation process is formally represented as follows:

$$score_{ori}(s) = \frac{f_{edges}(s) * f_{depth}(s) * f_{rounds}(s)}{f_{exec_time}(s)} \quad (3)$$

From this equation, the mutation score is inversely proportional to execution time, so a test case with large execution latency has fewer mutation opportunities. The more edges the test case covers, the more descendants mutated from it may be able to cover more edges and reveal potential vulnerabilities. The larger the number of test rounds, the later it is discovered and should be given more mutation opportunities. The deeper the path depth, the more likely it is to uncover vulnerabilities hidden deep in the program. For test cases in *exploit_queue*, VPGFUZZ should give more mutation opportunities to those with a higher likelihood of triggering vulnerabilities. Therefore, it is necessary to employ a mixed mutation score for the test cases. The calculation process is outlined as follows:

$$score_{mixed}(s) = score_{ori}(s) * (1 + f_{prob}(s)) \quad (4)$$

Here, $f_{prob}(s)$ denotes the probability of the test case s triggering vulnerabilities. When the probability exceeds a predefined threshold, we adopt a new mutation score $score_{mixed}$ to represent mutation times. Otherwise, the test case's mutation scores are still calculated in the original way.

V. EVALUATION

We conduct experiments to evaluate the effectiveness of VPGFUZZ in automated software testing. The evaluation aims to answer the following key research questions:

- RQ1: How effectively does VPGFUZZ identify potentially vulnerable paths? (§V-A)
- RQ2: How does VPGFUZZ outperform state-of-the-art fuzzers towards real-world programs? (§V-B)
- RQ3: What is the overhead of VPGFUZZ? (§V-C)
- RQ4: Can VPGFUZZ effectively discover zero-day vulnerabilities in real-world programs? (§V-D)

TABLE IV
TIME OF FIRST VULNERABILITY (TFV) FOR DIFFERENT TOOLS. HERE, THE SHORTEST TIME IS MARKED IN BLUE

	busybox	flvmeta ¹	jq	MP3gain	pdf2text	tcpdump	sqlite3	wav2swf ¹	objdump	pngimage ²	nasm ²	mujs ²
AFL	6.00h	<0.1h	0.44h	5.79h	5.45h	2.98h	0.11h	<0.1h	0.80h	–	–	–
AFLFast	–	<0.1h	–	1.42h	–	3.69h	0.11h	<0.1h	0.75h	–	–	–
TortoiseFuzz	func	–	<0.1h	–	2.59h	10.70h	1.44h	0.11h	<0.1h	10.89h	–	–
	loop	–	<0.1h	2.71h	2.83h	–	5.63h	0.11h	<0.1h	4.94h	–	–
	bb	8.99h	<0.1h	–	2.62h	10.40h	5.04h	0.11h	<0.1h	1.12h	–	–
CollAFL	3.85h	<0.1h	3.78h	1.12h	11.85h	2.87h	0.11h	<0.1h	1.09h	–	–	–
VPGFuzz	3.24h	<0.1h	0.36h	0.02h	4.60h	7.55h	0.11h	<0.1h	1.24h	–	–	–
Average	5.52h	–	1.82h	2.34h	8.60h	4.17h	0.11h	–	2.98h	–	–	–

¹ All fuzzers found the vulnerability within 1 minute.

² All fuzzers did not find any vulnerability within the specified time.

A. Benchmark Dataset

Following prior work [13], [14], [23], we select 12 popular real-world programs, as shown in Table IV, to serve as benchmarks for evaluating VPGFUZZ. Four programs, i.e., busybox, sqlite3, pdf2text, and pngimage, are not included in the training dataset, allowing us to test the system's generalization ability in detecting zero-day vulnerabilities for different programs. Although VPGFUZZ was trained using older versions of the remaining eight programs, which contained known vulnerabilities, we also evaluate VPGFUZZ using their newer versions. These updated versions have addressed many of the previous vulnerabilities, thereby testing the system's generalization ability in detecting zero-day vulnerabilities for the same program with different versions. Here, we do not consider test suites like LAVA-M [26] since its programs contain artificially inserted bugs, and their vulnerable path characteristics may differ from real-world ones.

B. Baseline Fuzzers

As VPGFUZZ primarily enhances the seed scheduling process with various path information, we compare its effectiveness with AFL [2], as well as AFLfast [12], CollAFL¹ and TortoiseFuzz [14], three state-of-the-art coverage-guided fuzzers that also optimize seed scheduling. AFL prioritizes seeds that execute quickly and have a smaller size. AFLfast [12] adopts a strategy of prioritizing seeds that trigger low-frequency paths. CollAFL [13] mitigates hash collisions in coverage-guided fuzzing and prioritizes seeds based on the number of memory access-related instructions as a heuristic for vulnerability exposure. TortoiseFuzz [14] defines a security-sensitive attribute for each edge and prioritizes seeds that hit more security-sensitive edges. Besides, TortoiseFuzz offers three modes for assessing the security sensitivity of edges at different levels: function, loop, and basic block. It then sorts the seeds based on these three metrics. As a result, we evaluate all three modes of TortoiseFuzz in our experiments. All experiments use the same initial seeds (i.e., test cases), mutation strategies, mutation dictionaries, and runtime parameters.

¹Since the official implementation of CollAFL is not publicly available, we use a third-party reproduction from GitHub: <https://github.com/batgui/collaf>

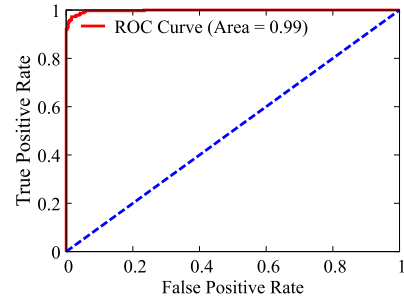


Fig. 8. ROC curve for recognizing vulnerable paths.

TABLE V
VULNERABLE PATH RECOGNITION CAPABILITY AND OVERHEAD FOR DIFFERENT MODELS

	Random Forest	Extra Trees	LightGBM	XGBoost	MLP	Logistic Regression
Accuracy	97.3%	96.7%	97.6%	98.1%	95.1%	77.9%
False Positive	2.9%	4.1%	2.5%	2.5%	4.8%	18.4%
False Negative	2.5%	2.5%	2.2%	1.3%	5.1%	25.9%
F1 Score	97.4%	96.7%	97.6%	98.1%	95.1%	77.1%
Inference Time (ms)	0.0205	0.0902	0.0685	0.0220	0.0030	0.0007
Memory Overhead (MB)	1.6791	9.2551	8.1311	4.6725	1.0587	0.0026

C. Experimental Environments

We run our experiments on a machine with 16 CPU cores (Intel(R) Xeon(R) Gold 6234 CPU @ 3.30GHz, 3.29 GHz), 64GB RAM, and 64-bit Ubuntu 18.04 LTS. Each fuzzing instance is assigned to a CPU core to prevent interference. Each experiment is run 5 times.

D. Effectiveness of Vulnerable Path Recognition (RQ1)

To evaluate the effectiveness of VPGFUZZ on recognizing vulnerable paths, we collect 3138 PoCs of vulnerabilities from CVE [15], OSS-Fuzz [1], and Github, and use them to generate vulnerable paths. As for non-vulnerable paths, we randomly select an equal number of paths from those where vulnerabilities have not yet been discovered, which are then partitioned into 90% for training and the remaining 10% for testing.

Vulnerable Path Recognition Capability and Overhead for Different Models. We conduct a comparative study across six canonical learning models, including Random Forest and Extra Trees (ensemble tree models), LightGBM and XGBoost (gradient-boosting frameworks), Multi-Layer Perceptron (a

TABLE VI

THE NUMBER OF UNIQUE PATHS TRIGGERED WITH DIFFERENT TOOLS

Program	AFL	AFLFast	TortoiseFuzz			CollAFL	VPGFuzz
			func	loop	bb		
busybox	567	640	599	646	675	616	605
flvmeta	391	323	287	368	347	409	349
jq	749	1231	785	1094	537	1060	529
MP3gain	287	331	296	289	269	314	275
pdf2text	2107	1596	1705	2106	1684	861	2016
tcpdump	1603	1778	2047	1749	1532	1668	1659
sqlite3	1531	1587	1575	1544	1616	1641	1516
wav2swf	178	230	192	223	150	186	175
objdump	1231	1272	575	862	1376	1184	1273
pngimage	255	235	255	274	272	268	259
nasm	4757	4618	4286	3220	4726	4488	3472
mujls	2753	3433	1965	1353	2727	2327	2320
Highest Count	2	4	1	1	2	2	0
				4			

TABLE VII

THE NUMBER OF DISCOVERED UNIQUE CRASHES WITH DIFFERENT TOOLS

Program	# of Unique Crashes						
	VPGFuzz	AFLFast	TortoiseFuzz			CollAFL	AFL
			func	loop	bb		
busybox	5	0	0	0	4	4	4
flvmeta	36	28	19	27	24	26	28
jq	28	1	0	5	0	2	17
MP3gain	7	2	2	2	1	3	5
pdf2text	2	0	2	0	2	1	2
tcpdump	13	8	1	11	4	9	10
sqlite3	53	37	53	46	46	48	44
wav2swf	20	19	7	21	14	16	17
objdump	14	16	4	8	16	13	17
Total	178	111	88	120	111	122	144
p-value of Wilcoxon signed-rank		0.009766	0.005999	0.003906	0.012008	0.001953	0.021451

neural network), and Logistic Regression, evaluating their effectiveness in recognizing vulnerable paths and their overhead. As shown in Table V, Random Forest, LightGBM, and XGBoost each achieve over 97% accuracy, with false positive rates below 3%, false negative rates below 2.5%, inference times under 1 ms, and memory overheads under 10 MB. Any of these three models can serve as a practical choice for vulnerable path recognition. Since classifier design is not the core contribution of our work, we adopt Random Forest as the default classifier in VPGFUZZ for identifying vulnerable paths. Figure 8 shows the RoC (Receiver Operating Characteristic) curve of selected model. We observe that the AUC (Area Under the ROC Curve) value is approximately 99.8%, indicating that VPGFUZZ is effective in recognizing vulnerable paths.

Path Recognition Model Generalizability. In our testing samples, *sqlite3*, *pdf2text*, and *busybox* are not included in the training dataset and differ significantly from the training programs. Specifically: (1) *sqlite3* is a lightweight database engine, while the training set contains no database or SQL-related software. (2) *pdf2text* extracts text from PDFs, in contrast to *tiff2pdf* in the training set, which converts TIFF files to PDFs without performing text parsing. The two differ in input format, output structure, and underlying libraries. (3) Although both *busybox* (test) and *binutils* (train) are command-line utility suites, they differ in design. *busybox*

employs a monolithic applet-based architecture for general-purpose commands, whereas *binutils* consists of standalone tools specialized for binary processing. Despite these substantial structural differences, the model still enables VPGFuzz to efficiently uncover vulnerabilities in all three unseen testing programs, as shown in Table IV and Table VIII. These results demonstrate the method’s ability to generalize across diverse software domains.

Answer to RQ1. Our VPGFUZZ can effectively identify vulnerable paths with a high AUC value of 99.8%.

E. Comparison With State-of-the-Art (RQ2)

We compare VPGFUZZ with existing fuzzers in three aspects: time to first vulnerability, code coverage, and triggered crashes.

1) *Time to First Vulnerability:* Time to First Vulnerability (TFV) refers to the time that a fuzzer takes to discover a vulnerability in the target program. Prior studies [17], [27] apply time to first crash (TFC) as a valuable metric for assessing the efficacy of fuzzers. However, a crash, such as an assertion failure, may not necessarily indicate a security issue. Therefore, we propose using TFV instead of TFC to provide a more accurate assessment of fuzzer effectiveness in identifying security vulnerabilities. Once a crash happens, we would manually check if it is caused by vulnerability. Table IV shows TFV for all fuzzers. Here, we focus on the experimental results from 7 programs since other programs either do not trigger crashes or their TFV is extremely short, *e.g.*, less than 0.1 hours. Compared to other fuzzers, we observe VPGFuzz achieves the fastest TFV for 5 out of the 7 programs.

While TortoiseFuzz and AFLfast show the faster TFV for two other programs, our work exhibits significant advantages over them in most tasks. For instance, when utilizing VPGFUZZ to fuzz Busybox, we observed a vulnerability within 3.24 hours, which is 5 hours earlier than TortoiseFuzz. Remarkably, under identical experimental conditions, VPGFUZZ revealed vulnerabilities in all programs tested, while AFLfast and three modes of TortoiseFuzz failed to discover any vulnerabilities in some cases. Notably, it is highly effective to use VPGFUZZ to fuzz MP3gain. For this program, VPGFUZZ triggers the first vulnerability in less than 2 minutes, which is 117 times faster than the average TFV.

2) *Code Coverage:* Code coverage is often used as a key metric to reveal the capability of fuzzers exploring code. Intuitively, more explored code indicates a higher likelihood of vulnerability discovery. However, code coverage is not exactly equivalent to the number of discovered vulnerabilities, since explored code is sometimes irrelevant to vulnerabilities, leading to unnecessary computational overhead. Therefore, rather than improving code coverage, our VPGFUZZ focuses on exploring the paths that may trigger vulnerabilities, and prioritizes the corresponding test cases in the testing process. Table VI shows the code coverage of each fuzzer after running on 12 programs for 12 hours. Here, we use the number of unique explored paths to represent code coverage.

Compared to the baseline fuzzer AFL, we observe that our VPGFUZZ and three advanced fuzzers, *i.e.*, AFLfast, TortoiseFuzz and CollAFL, do not show a significant advantage

TABLE VIII
THE NUMBER OF VULNERABILITIES, AND THE TYPE OF VULNERABILITIES FOUND BY DIFFERENT TOOLS

Program	# of Vulnerabilities ¹							Vulnerability Type ²						
	AFL	AFLFast	TortoiseFuzz			CollAFL	VPGFuzz	AFL	AFLFast	TortoiseFuzz			CollAFL	VPGFuzz
			func	loop	bb					func	loop	bb		
busybox	2	0	0	0	1	2	3	UAF, HBO	—	—	—	UAF	HBO	UAF, HBO
flvmeta	2	2	2	2	2	2	2	SEGV	SEGV	SEGV	SEGV	SEGV	SEGV	SEGV
jq	1	0	0	1	0	1	1	HBO	—	—	HBO	—	HBO	HBO
MP3gain	3	2	2	2	1	3	3	HBO, SBO	HBO	HBO	HBO	HBO	HBO, SBO	HBO, SBO
pdf2text	1	0	0	0	0	0	1	HBO	—	—	—	—	—	HBO
tcpdump	7	4	1	9	1	8	11	HBO	HBO	HBO	HBO	HBO	HBO	HBO
sqlite3	1	1	1	1	1	1	1	MLK	MLK	MLK	MLK	MLK	MLK	MLK
wav2swf	8	9	4	10	6	5	8	HBO, FPE	HBO, FPE	HBO, FPE	HBO, UAF, FPE	HBO, FPE	HBO, FPE	HBO, UAF, FPE
objdump	3	3	1	2	2	3	2	HBO, SEGV	HBO, SEGV	SEGV	HBO, SEGV	HBO, SEGV	HBO, SEGV	HBO, SEGV
Total	28	21	11		27	14	25	6	4	4		5	5	6

¹ The vulnerabilities found by 3 modes of TortoiseFuzz overlap, so the final number is their union.

² UAF: use-after-free, HBO: heap-buffer-overflow, SEGV: segmentation fault, SBO: stack-buffer-overflow, MLK: memory leak, FPE: Floating Point Exception.

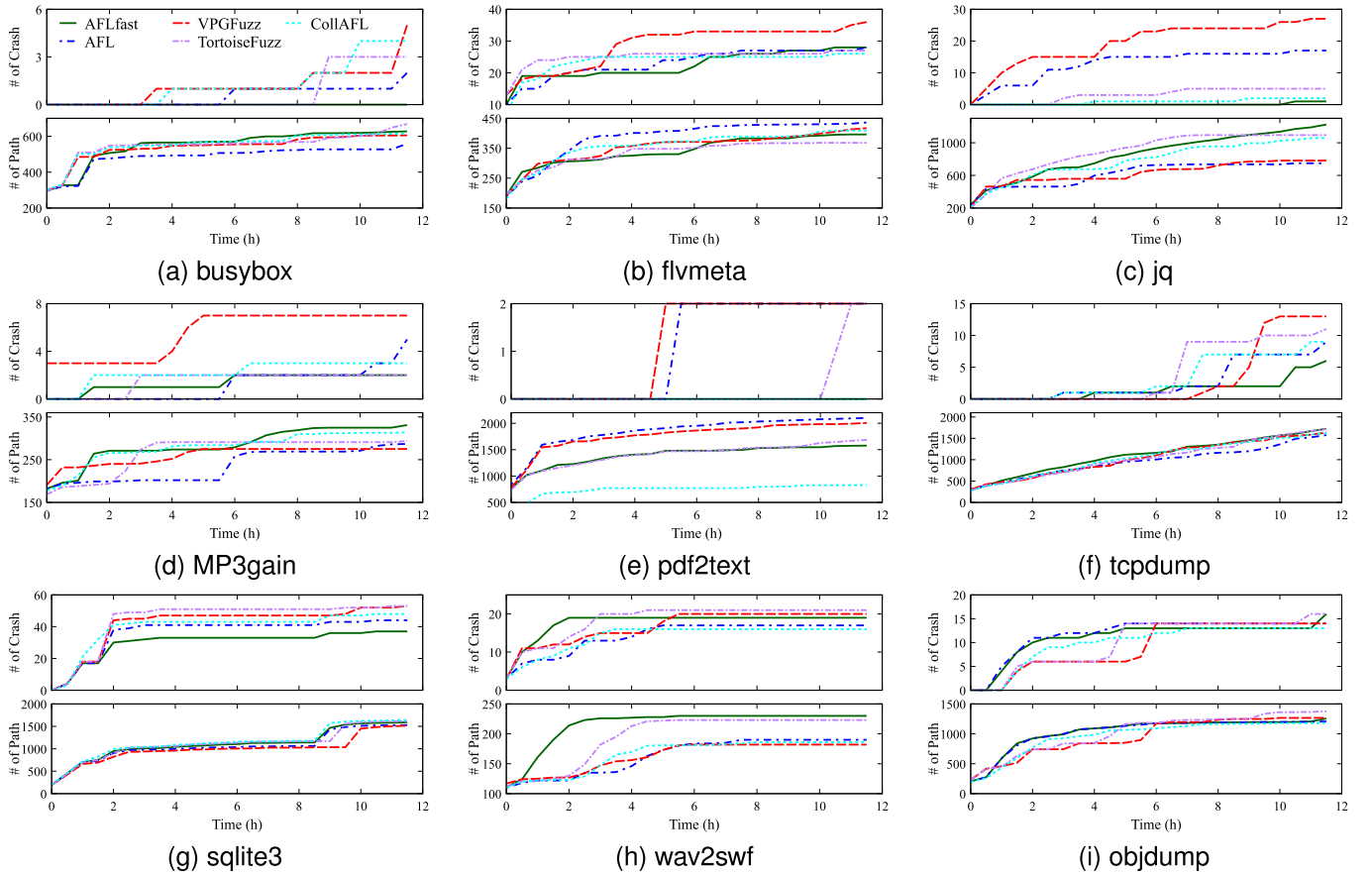


Fig. 9. Trends in path coverage and the number of triggered crashes over time.

in terms of code coverage. Here, we emphasize that high code coverage does not mean high likelihood of vulnerability discovery. This is because covered code during fuzzing is sometimes irrelevant to vulnerabilities, leading to unnecessary computational overhead. Therefore, while VPGFUZZ does not achieve high code coverage, we do not consider this as a drawback. In fact, VPGFUZZ focuses on exploring the paths that likely trigger vulnerabilities rather than triggering more code. In some cases, this strategy may help find more vulnerabilities (Table VIII).

3) *Triggered Crashes Vs. Code Coverage*: Furthermore, we conduct experiments to investigate the relationship between code coverage and crashes. Figure 9 shows the change of two fuzzing properties over time: the number of triggered crashes and code coverage. For TortoiseFuzz, we choose the mode that triggers the most crashes for comparison. While VPGFUZZ does not achieve the best code coverage for all programs, it triggers the largest number of crashes for 7 programs, and it can expose crashes in the very earlier phase. Table VII provides a detailed comparison of the unique crashes

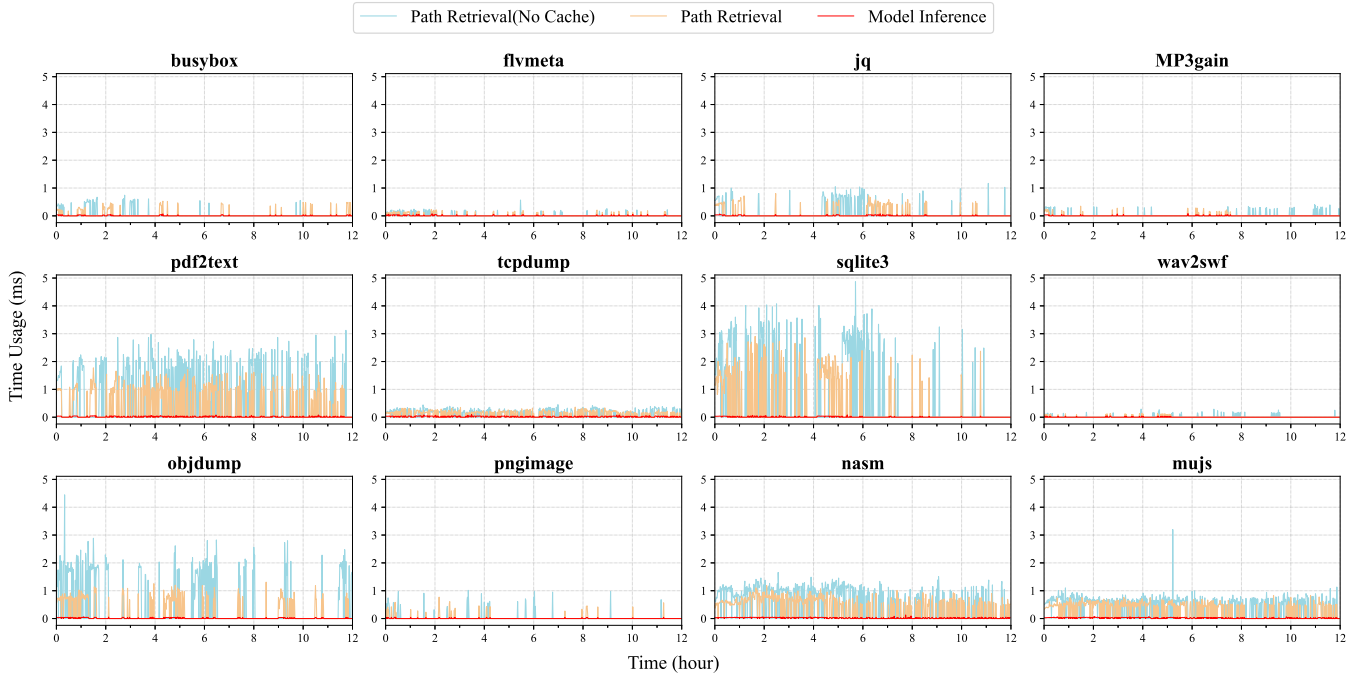


Fig. 10. Time overhead in path retrieval and model inference.

triggered by different tools. We perform paired Wilcoxon signed-rank tests between VPGFUZZ and each baseline fuzzer. As shown in Table VII, all p-values are below 0.05, indicating that VPGFUZZ’s advantage in triggering unique crashes is statistically significant.

Although an increase in unique crashes does not necessarily imply a greater number of vulnerabilities, from Table VIII, we can see that among the 7 programs with the largest number of crashes exposed by VPGFUZZ, it also finds the largest number of vulnerabilities. In terms of the total count, compared to other fuzzers, VPGFUZZ finds the largest number of crashes and vulnerabilities, covering the most comprehensive range of vulnerability types.

Answer to RQ2. Compared to existing coverage-guided fuzzers, VPGFUZZ achieves better performance for most programs in terms of the number of vulnerabilities and the speed of vulnerability exposure.

F. Performance Overhead (RQ3)

For discovering more vulnerabilities in a short time, VPGFUZZ introduces additional overhead, such as path retrieval and vulnerable path model inference, which may impact its performance compared to AFL. To evaluate this cost, we measured the runtime overhead on the benchmark programs, focusing on two key components: (i) the latency of path retrieval, both with and without our second-level caching mechanism, and (ii) the latency of model inference. As for mutation overhead, since VPGFUZZ adopts AFL’s default mutation strategy, we do not measure the mutation runtime overhead.

Given that the fuzzer executes a large number of test cases per second, we aggregate the total overhead over 60-second intervals and compute the average per-execution latency within

TABLE IX
FUZZING THROUGHPUT COMPARISON

Program	Throughput(m)					
	AFL	AFLFast	TortoiseFuzz		CollAFL	VPGFuzz
			func	loop	bb	
busybox	5.1	5.63	3.79	4.05	5.04	5.70
flvmeta	22.9	19.45	16.4	19.4	17.7	18.7
jq	3.78	4.48	3.68	4.3	3.52	4.60
MP3gain	7.28	8.53	11.29	11.1	10.9	9.82
pdf2text	1.93	2.8	3.17	3.51	3.06	1.61
tcpdump	9.84	9.09	9.07	10.4	10	10.10
sqlite3	1.18	1.81	1.79	1.22	1.18	1.18
wav2swf	17.7	22.05	8.98	14	4.63	18.00
objdump	1.85	1.88	0.62	3.52	2.2	1.24
			6.53	7.94	6.47	
Average	7.95	8.41	6.98		7.88	7.1
	-	+5.79%	-12.20%		-0.88%	-10.69%

each window. To evaluate inference overhead, we use *m2cgen* to transpile the trained model into C code, which is then compiled into the fuzzer binary. The reported inference latency corresponds to this compiled version. The compiled model occupies approximately 2 MB of resident memory, including 1.8 MB in the `.text` segment and 0.16 MB in the `.rodata` segment. Importantly, since all model weights are embedded as compile-time constants, each inference invocation requires zero additional heap memory allocation.

Figure 10 presents the results across all target programs. As shown, the model inference overhead is negligible, typically remaining below 0.05 ms. In contrast, path retrieval incurs slightly higher latency. However, our second-level cache effectively mitigates this cost. For larger programs such as *sqlite3*, *pdf2text*, and *objdump*, the cached version achieves a significant reduction in path retrieval latency compared to the uncached baseline.

TABLE X
PREVIOUSLY UNKNOWN VULNERABILITIES IDENTIFIED BY VPGFUZZ

Program	Version	Vulnerability Type	ID	Status	Severity Level
BusyBox	1.36.1	Heap Buffer Overflow	CVE-2023-42366	Assigned	High
		Use After Free	CVE-2023-42365	Assigned	High
		Use After Free	CVE-2023-42364	Assigned	High
		Use After Free	CVE-2023-42363	Assigned	High
		Use After Free	CVE-2023-50678	Unpublished	-
		Use After Free	CNVD-2024-07285	Assigned	Medium
		Heap Buffer Overflow	-	Request	-
jq	1.7-37-g88f01a7	Heap Buffer Overflow	CVE-2023-49355	Assigned	High
mp3gain	1.6.2	Stack Buffer Overflow	CVE-2023-49356	Assigned	High
		Segmentation Fault	CNNVD-2023-31029573	Assigned	Low
	2.16.02rc5	Memory Leak	CNNVD-2023-60081157	Assigned	Low
		Heap Buffer Overflow	CNNVD-2023-04348213	Assigned	Low
		Use After Free	CNNVD-2023-62399592	Assigned	Low
		Heap Buffer Overflow	CVE-2023-50679	Unpublished	-
		Memory Leak	CVE-2023-50680	Unpublished	-
nasm	2.16.01	Memory Leak	CVE-2023-50681	Unpublished	-
		Memory Leak	CNVD-2024-02641	Assigned	Medium
		Heap Buffer Overflow	-	Request	-
		mujs	1.3.4	Resource Exhaustion	-
objdump	2.41	Memory Leak	CNVD-2024-08948	Unpublished	-
		Memory Leak	CNVD-2024-08949	Unpublished	-
tcpdump	4.8.1	Use After Free	-	Fixed	-
	4.9.1	Heap Buffer Overflow	-	Fixed	-
		Heap Buffer Overflow	-	Fixed	-
SUM		10 CVEs,4 CNNVDs, 4 CNVDs		24	6 Severs

We also assess the performance overhead through fuzzing throughput. Table IX presents the throughput of four fuzzers testing 9 distinct programs. For TortoiseFuzz, we take the average throughput of 3 modes for comparison. Compared to AFL, VPGFUZZ's average throughput decreased by 10.69%, which is acceptable in practice.

Answer to RQ3. While VPGFUZZ introduces extra overhead, the average throughput of VPGFUZZ is 10.69% lower than AFL and 1.72% higher than TortoiseFuzz. Such overhead is acceptable and would not affect final fuzzing results.

G. Effectiveness of Discovering Real-World 0-Day Vulnerabilities (RQ4)

Table X shows a collection of real-world unknown vulnerabilities discovered by our VPGFUZZ through experiments. Here, we enumerate each identified vulnerability along with its corresponding program, vulnerability type, assigned vulnerability ID, current status, and severity level. In total, VPGFUZZ unveils 24 distinct unknown vulnerabilities of 6 different types in 7 programs, including buffer overflow, use-after-free, stack overflow, memory leaks, resource exhaustion, and segmentation faults. Notably, 6 of these vulnerabilities have received a CVSS score exceeding 7.0, indicating their potential to lead to severe consequences, *e.g.*, arbitrary code execution. Among the 24 identified vulnerabilities, 10 have received CVE IDs, 4 have obtained CNVD IDs, and 4 have been assigned CNNVD IDs. This indicates VPGFUZZ can effectively discover zero-day vulnerabilities from real-world programs.

We conduct two in-depth case studies on real-world CVEs (CVE-2023-49356 and CVE-2023-50678), as shown in Table X, to further illustrate how our method discovers vulnerabilities more effectively than existing approaches.

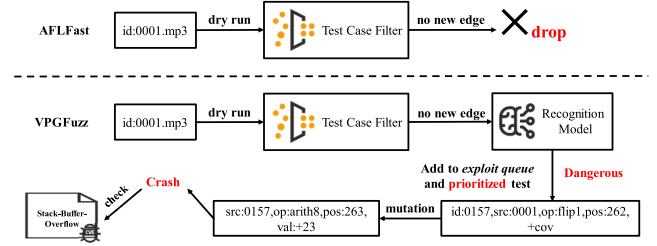


Fig. 11. Case Study: Stack-buffer-overflow in MP3gain (CVE-2023-49356), found by VPGFUZZ, missed by AFLFast.

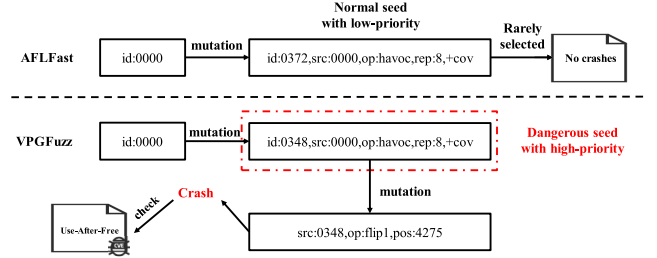


Fig. 12. Case Study: Use-after-free in BusyBox (CVE-2023-50678), found by VPGFUZZ, missed by AFLFast.

Case Study 1: Stack Buffer Overflow in MP3gain (CVE-2023-49356). As shown in Figure 11, although both AFLFast and VPGFUZZ start with the same initial seed, *id:0001.mp3*, they process it differently. After the dry-run phase, the seed does not yield any new edge coverage. As a result, AFLFast discards it according to its default fitness function. In contrast, VPGFUZZ feeds the path information of the same seed into the recognition model, which identifies it as vulnerable. The seed is then added to the *exploit_queue* and assigned high priority. We observe that a test case derived from *id:0157* eventually triggers a program crash, which can be traced back to the initial seed *id:0001.mp3*. Further analysis of the crash reveals a stack buffer overflow vulnerability, which has been assigned CVE-2023-49356. Our findings show that VPGFUZZ uncovers the first vulnerability in MP3Gain 70 times faster than existing fuzzers by recognizing and prioritizing the initial seed *id:0001.mp3* as potentially vulnerable—while other fuzzers discard it during the dry-run phase.

Case Study 2: Use After Free in busybox (CVE-2023-50678). As shown in Figure 12, although the two seeds—*id:0372* and *id:0348*—are both generated from the same initial seed using identical mutation operations and introduce new edge coverage, AFLFast and VPGFUZZ handle them differently. Specifically, AFLFast does not consider *id:0372* as favored and therefore assigns it a very low probability of being selected in subsequent fuzzing iterations. As a result, AFLFast fails to discover the vulnerability during its testing period. In contrast, VPGFUZZ recognizes that *id:0348* is likely to trigger a vulnerability and prioritizes it during fuzzing. In the subsequent fuzzing process, a test case derived from *id:0348* triggers a program crash. Root-cause analysis of the crash reveals a use-after-free vulnerability, which has been assigned CVE-2023-50678.

These case studies demonstrate that the vulnerable path guidance improvements introduced by VPGFUZZ in test case filtering and seed selection are effective, enabling it to uncover more vulnerabilities.

Answer to RQ4. VPGFUZZ find 24 previously unknown vulnerabilities, demonstrating that VPGFUZZ can effectively find zero-day vulnerabilities from real-world programs. Additionally, case studies analysis demonstrates that VPGFUZZ can avoid delaying or missing potential vulnerabilities.

VI. RELATED WORK

Coverage-guided fuzzing is proposed to maximize code coverage during testing [28], [29]. AFL [2] and libfuzzer [4] leverage lightweight instrumentation to track code coverage and employ various mutation strategies to generate malformed test cases. To enhance effectiveness, symbolic execution [30], [31], [32], [33], taint analysis [34], [35], [36], and machine learning [19], [21], [37], [38], [39], [40], [41], [42] based techniques are proposed. Moreover, tailored solutions are developed for specific targets, i.e., protocol [6], [7], [43], [44], OS kernels [8], [9], [45], [46], [47], firmware [10], [11], [48], [49], [50], neural networks [51], [52], [53], smart contracts [54], [55], [56], and autonomous driving [57]. Orthogonal to existing approaches that are mainly for improving test case generation, seed scheduling, and status monitoring, our study focuses on optimizing the seed scheduling phase. For a comprehensive overview of fuzzing research and trends, see the recent survey [58].

Coverage metric is measured by calculating the number of executed basic blocks in Sidewinder [28] and the improved edge coverage in AFL. VUzzer [34] prioritize common basic blocks over error-handling basic blocks, Angora [35] introduces calling context in addition to edge coverage, and MemFuzz [59] adopts the number of memory access addresses as a metric. To solve the hash conflicts, CollAFL [13] uses three fine-grained path attributes to ensure each edge has a unique hash value. Different metrics are also proposed for specific testing requirements. For instance, MemLock [60] considers memory consumption to discover resource exhaustion vulnerabilities. SlowFuzz [61], PerfFuzz [62], and ReScue [63] analyze time overhead to uncover algorithmic complexity vulnerabilities. IJON [64] incorporate human's feedback on program state to the fuzzer.

Seed selection is simply conducted in order by Honggfuzz [5]. AFL gives priority to seeds with faster execution speed and smaller size to enable more tests. VUzzer [34] prioritizes seeds with deeper execution paths to explore deeply hidden vulnerabilities. FairFuzz [65] prefers seeds that can hit branches covered by a small number of inputs and continue to hit rare branches by mutation. AFLfast [12] models the fuzzing process as a Markov chain to select low-frequency paths and less-selected seeds. Directed fuzzing [39], [66], [67], [68] targets specific code regions within a program, prioritizing seeds whose execution paths are nearest to the designated area.

Several fuzzing strategies have proposed the explore-exploit trade-off to improve efficiency. EcoFuzz [16] and AFL-Hier [17] both adopt multi-armed bandit algorithms to optimize

seed scheduling. EcoFuzz prioritizes seeds with low self-transition probability to favor exploration, while AFL-Hier organizes seeds into hierarchical levels to achieve multi-granularity coverage. Entropic [18] treats fuzzing as a species discovery problem and uses Shannon entropy [69] to estimate a seed's potential to uncover novel behaviors. These methods are built on the coverage-guided fuzzing paradigm, where exploration and exploitation are directed by coverage metrics. In contrast, our approach is guided by vulnerable paths: we estimate each seed's likelihood of triggering vulnerabilities and allocate resources accordingly. While prior strategies aim to avoid premature convergence to frequently covered paths, our work aims to reduce the distinct risk: they may overcommit to high-confidence vulnerable paths too early, neglecting other inputs with lower confidence but potentially vulnerable. To address this, our explore-exploit strategy balances predicted vulnerability likelihood with the discovery of new code coverage, ensuring both efficient exploitation and broad exploration.

She et al. [70] chooses seeds with a higher centrality score. To select seeds related with security issues, CollAFL [13] and TortoiseFuzz [14] propose rules regarding the memory access/untouched branch/untouched descendant properties and specific function/loop/basic block, respectively. However, this process requires huge human expertise and manually defined rules are error-prone. In contrast, our learning based approach automatically learns from existing real-world vulnerabilities, enabling more effective vulnerable seed selection. Distinct from the aforementioned works, our approach introduces a key innovation by leveraging known vulnerability knowledge to enhance both seed selection and test case filtering. Specifically, we prioritize seeds that are likely to trigger vulnerabilities during seed selection and allocate them more mutation energy, thereby enabling faster vulnerability discovery. At the same time, our vulnerable path recognition model allows us to retain inputs that may not cover new edges but still have the potential to expose vulnerabilities, as illustrated in the motivating example (Section II-A). This dual enhancement not only accelerates vulnerability exposure but also reduces the likelihood of missing potential vulnerabilities, as demonstrated by the case studies in Section V-D, which validate the effectiveness of our method in both aspects.

VII. DISCUSSION

While our vulnerable path recognition model demonstrates reasonable generalization in our evaluation, we acknowledge that its precision and fuzzing effectiveness may degrade when applied to previously unseen or significantly different code-bases. Such scenarios can drive execution into input and state spaces that lie outside the training distribution.

To address this limitation, a potential solution is to incorporate an online learning mechanism that incrementally updates the recognition model based on feedback from the fuzzing process. For example, inputs that consistently lead to false predictions or reveal new behavioral patterns could be used to refine the model, allowing it to continuously adapt to out-of-distribution execution paths. This approach is consistent with prior work such as Neuzz [42], which integrates online learning into fuzzing by continuously updating a neural network

approximation of program behavior, enabling the fuzzer to adapt its mutation strategy based on observed feedback.

VIII. CONCLUSION

In this paper, we propose VPGFUZZ, a vulnerable path-guided fuzzer that leverages known vulnerability path knowledge to enhance the performance of fuzzing. First, we build a vulnerable path recognition model and employ it to help our fuzzer identify those ‘danger’ seeds that may trigger vulnerabilities even though they do not trigger any new edge during execution. For these seeds, we design a new seed selection strategy, to allow the fuzzer to focus more on these seeds and uncover vulnerabilities faster. Also, this strategy ensures that our fuzzer can explore more code to find unknown vulnerabilities. Experiments show VPGFUZZ discover 24 previously unknown vulnerabilities. Compared to existing fuzzers, VPGFUZZ achieves significantly faster vulnerability discovery speeds, ranging from approximately 1.2 to 70 times faster across most programs.

ACKNOWLEDGMENT

Any opinions, findings, conclusions, or recommendations expressed in this material are those of the authors and do not necessarily reflect the views of NSFC and other sponsors.

REFERENCES

- [1] Google.(2016). *OSS-Fuzz: Continuous Fuzzing for Open Source Software*. [Online]. Available: <https://github.com/google/oss-fuzz>
- [2] M. Zalewski. (2013). *American Fuzzy Lop (2.52b)*. [Online]. Available: <https://lcamtuf.coredump.cx/afl/>
- [3] A. Fioraldi, D. Maier, H. Eibfeldt, and M. Heuse, “AFL+: Combining incremental steps of fuzzing research,” in *Proc. 14th USENIX Workshop Offensive Technol. (WOOT 20)*, 2020, pp. 1–12.
- [4] LLVM. *Libfuzzer—A Library for Coverage-Guided Fuzz Testing*. Accessed: Sep. 9, 2025. [Online]. Available: <https://llvm.org/docs/LibFuzzer.html#margin>
- [5] Google.*Honggfuzz*. Accessed: Sep. 9, 2025. [Online]. Available: <https://github.com/google/honggfuzz>
- [6] V.-T. Pham, M. Böhme, and A. Roychoudhury, “AFLNET: A greybox fuzzer for network protocols,” in *Proc. IEEE 13th Int. Conf. Softw. Test., Validation Verification (ICST)*, Oct. 2020, pp. 460–465.
- [7] Z. Luo et al., “Bleem: Packet sequence oriented fuzzing for protocol implementations,” in *Proc. 32nd USENIX Secur. Symp. (USENIX Secur. 23)*, 2023, pp. 4481–4498.
- [8] J. Corina et al., “DIFUZE: Interface aware fuzzing for kernel drivers,” in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, New York, NY, USA, Oct. 2017, pp. 2123–2138.
- [9] S. Schumilo, C. Aschermann, R. Gawlik, S. Schinzel, and T. Holz, “KAFL: Hardware-assisted feedback fuzzing for OS kernels,” in *Proc. 26th USENIX Secur. Symp.*, 2017, pp. 167–182.
- [10] X. Feng et al., “Snipuzz: Black-box fuzzing of IoT firmware via message snippet inference,” in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Nov. 2021, pp. 337–350.
- [11] Y. Zheng, A. Davanian, H. Yin, C. Song, H. Zhu, and L. Sun, “FIRM-AFL: High-throughput greybox fuzzing of IoT firmware via augmented process emulation,” in *Proc. 28th USENIX Secur. Symp.*, 2019, pp. 1099–1114.
- [12] M. Böhme, V.-T. Pham, and A. Roychoudhury, “Coverage-based greybox fuzzing as Markov chain,” in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Oct. 2016, pp. 1032–1043.
- [13] S. Gan et al., “CollAFL: Path sensitive fuzzing,” in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2018, pp. 679–696.
- [14] Y. Wang et al., “Not all coverage measurements are equal: Fuzzing by coverage accounting for input prioritization,” in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2020, pp. 1–17.
- [15] The MITRE Corporation. *Common Vulnerabilities and Exposures*. Accessed: Sep. 9, 2025. [Online]. Available: <https://www.cve.org/>
- [16] T. Yue et al., “EcoFuzz: Adaptive energy-saving greybox fuzzing as a variant of the adversarial multi-armed bandit,” in *Proc. 29th USENIX Secur. Symp. (USENIX Secur.)*, Aug. 2020, pp. 2307–2324.
- [17] J. Wang, C. Song, and H. Yin, “Reinforcement learning-based hierarchical seed scheduling for greybox fuzzing,” in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2021, pp. 1–17.
- [18] M. Böhme, V. J. M. Manès, and S. K. Cha, “Boosting fuzzer efficiency: An information theoretic perspective,” in *Proc. 28th ACM Joint Meeting Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng.*, Nov. 2020, pp. 678–689.
- [19] Y. Wang, Z. Wu, Q. Wei, and Q. Wang, “NeuFuzz: Efficient fuzzing with deep neural network,” *IEEE Access*, vol. 7, pp. 36340–36352, 2019.
- [20] N. Nethercote and J. Seward, “Valgrind: A framework for heavyweight dynamic binary instrumentation,” *ACM SIGPLAN Notices*, vol. 42, no. 6, pp. 89–100, Jun. 2007.
- [21] L. Joffe and D. Clark, “Directing a search towards execution properties with a learned fitness function,” in *Proc. 12th IEEE Conf. Softw. Test., Validation Verification (ICST)*, Apr. 2019, pp. 206–216.
- [22] N. Redini et al., “Karonte: Detecting insecure multi-binary interactions in embedded firmware,” in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2020, pp. 1544–1561.
- [23] Y. Li et al., “Unifuzz: A holistic and pragmatic metrics-driven platform for evaluating fuzzers,” in *Proc. USENIX Secur. Symp.*, 2021, pp. 2777–2794.
- [24] Z. Li et al., “VulDeePecker: A deep learning-based system for vulnerability detection,” 2018, *arXiv:1801.01681*.
- [25] A. J. Smith, “Cache memories,” *ACM Comput. Surveys (CSUR)*, vol. 14, no. 3, pp. 473–530, 1982.
- [26] B. Dolan-Gavitt et al., “LAVA: Large-scale automated vulnerability addition,” in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2016, pp. 110–121.
- [27] M. Böhme and B. Falk, “Fuzzing: On the exponential cost of vulnerability discovery,” in *Proc. 28th ACM Joint Meeting Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng.*, Nov. 2020, pp. 713–724.
- [28] S. Embleton, S. Sparks, and R. Cunningham, “Sidewinder: An evolutionary guidance system for malicious input crafting,” Black hat, Las Vegas, NV, USA, Tech. Rep., 2006.
- [29] J. D. DeMott and R. Enbody, “Revolutionizing the field of grey-box attack surface testing with evolutionary fuzzing,” BlackHat Defcon, Las Vegas, NV, USA, Tech. Rep., 2007.
- [30] N. Stephens et al., “Driller: Augmenting fuzzing through selective symbolic execution,” in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2016, pp. 1–16.
- [31] I. Yun, S. Lee, M. Xu, Y. Jang, and T. Kim, “QSYM: A practical concolic execution engine tailored for hybrid fuzzing,” in *Proc. 27th USENIX Secur. Symp.*, 2018, pp. 745–761.
- [32] Y. Chen et al., “SAVIOR: Towards bug-driven hybrid testing,” in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2020, pp. 1580–1596.
- [33] H. Peng, Y. Shoshitaishvili, and M. Payer, “T-fuzz: Fuzzing by program transformation,” in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2018, pp. 697–710.
- [34] S. Rawat et al., “VUZZer: Application-aware evolutionary fuzzing,” in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2017, pp. 1–14.
- [35] P. Chen and H. Chen, “Angora: Efficient fuzzing by principled search,” in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2018, pp. 711–725.
- [36] C. Aschermann, S. Schumilo, T. Blazytko, R. Gawlik, and T. Holz, “REDQUEEN: Fuzzing with input-to-state correspondence,” in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2019, pp. 1–15.
- [37] P. Godefroid, H. Peleg, and R. Singh, “Learn&fuzz: Machine learning for input fuzzing,” in *Proc. 32nd IEEE/ACM Int. Conf. Automated Softw. Eng. (ASE)*, Oct. 2017, pp. 50–59.
- [38] J. Wang, B. Chen, L. Wei, and Y. Liu, “Skyfire: Data-driven seed generation for fuzzing,” in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2017, pp. 579–594.
- [39] P. Zong, T. Lv, D. Wang, Z. Deng, R. Liang, and K. Chen, “FuzzGuard: Filtering out unreachable inputs in directed grey-box fuzzing through deep learning,” in *Proc. 29th USENIX Secur. Symp. (USENIX Secur. 20)*, 2020, pp. 2255–2269.
- [40] X. Liu, X. T. Li, R. Prajapati, and D. H. Wu, “DeepFuzz: Automatic generation of syntax valid C programs for fuzz testing,” in *Proc. 33rd AAAI Conf. Artif. Intell. 31st Innov. Appl. Artif. Intell. Conf. 9th AAAI Symp. Educ. Adv. Artif. Intell.*, 2019, pp. 1044–1051.
- [41] D. She, K. Pei, D. Epstein, J. Yang, B. Ray, and S. Jana, “NEUZZ: Efficient fuzzing with neural program smoothing,” in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2019, pp. 803–817.

- [42] S. Lee, H. Han, S. K. Cha, and S. Son, "Montage: A neural network language model-guided JavaScript engine fuzzer," in *Proc. 29th USENIX Secur. Symp.*, Aug. 2020, pp. 2613–2630.
- [43] R. S. P. Amini and A. Portnoy. *Boofuzz: Network Protocol Fuzzing for Humans*. Accessed: Sep. 9, 2025. [Online]. Available: <https://github.com/jtpereyda/boofuzz>
- [44] M. Eddington. *Peach Fuzzer Community Edition V3*. Accessed: Sep. 9, 2025. [Online]. Available: <https://gitlab.com/peachtech/peach-fuzzer-community>
- [45] Google.Syzkaller—Kernel Fuzzer. Accessed: Sep. 9, 2025. [Online]. Available: <https://github.com/google/syzkaller>
- [46] T. Yin, Z. Gao, Z. Xiao, Z. Ma, M. Zheng, and C. Zhang, "KextFuzz: Fuzzing macOS kernel EXTensions on apple silicon via exploiting mitigations," in *Proc. 32nd USENIX Secur. Symp. (USENIX Secur. 23)*, 2023, pp. 5039–5054.
- [47] M. Cho, D. An, H. Jin, and T. Kwon, "BoKASAN: Binary-only kernel address sanitizer for effective kernel fuzzing," in *Proc. 32nd USENIX Secur. Symp. (USENIX Secur. 23)*, 2023, pp. 4985–5002.
- [48] J. Chen et al., "IoTfuzzer: Discovering memory corruptions in IoT through app-based fuzzing," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2018, pp. 1–15.
- [49] M. Muench, J. Stijohann, F. Kargl, A. Francillon, and D. Balzarotti, "What you corrupt is not what you crash: Challenges in fuzzing embedded devices," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2018, pp. 1–15.
- [50] Z. Gui, H. Shu, F. Kang, and X. Xiong, "FIRMCORN: Vulnerability-oriented fuzzing of IoT firmware via optimized virtual execution," *IEEE Access*, vol. 8, pp. 29826–29841, 2020.
- [51] J. Guo, Y. Jiang, Y. Zhao, Q. Chen, and J. Sun, "DLFuzz: Differential fuzzing testing of deep learning systems," in *Proc. 26th ACM Joint Meeting Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng.*, Oct. 2018, pp. 739–743.
- [52] K. Pei, Y. Cao, J. Yang, and S. Jana, "DeepXplore: Automated whitebox testing of deep learning systems," in *Proc. 26th Symp. Operating Syst. Princ.*, Oct. 2017, pp. 1–18.
- [53] X. Xie et al., "DeepHunter: A coverage-guided fuzz testing framework for deep neural networks," in *Proc. 28th ACM SIGSOFT Int. Symp. Softw. Test. Anal.*, Jul. 2019, pp. 146–157.
- [54] J. He, M. Balunović, N. Ambroladze, P. Tsankov, and M. Vechev, "Learning to fuzz from symbolic execution with application to smart contracts," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2019, pp. 531–548.
- [55] T. D. Nguyen, L. H. Pham, J. Sun, Y. Lin, and Q. T. Minh, "SFuzz: An efficient adaptive fuzzer for solidity smart contracts," in *Proc. IEEE/ACM 42nd Int. Conf. Softw. Eng. (ICSE)*, Oct. 2020, pp. 778–788.
- [56] F. Ma et al., "LOKI: State-aware fuzzing framework for the implementation of blockchain consensus protocols," in *Proc. Netw. Distrib. Syst. Secur. Symp.*, 2023, pp. 1–18.
- [57] S. Kim, M. Liu, J. J. Rhee, Y. Jeon, Y. Kwon, and C. H. Kim, "DriveFuzz: Discovering autonomous driving bugs through driving quality-guided fuzzing," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Nov. 2022, pp. 1753–1767.
- [58] X. Zhu, S. Wen, S. Camtepe, and Y. Xiang, "Fuzzing: A survey for roadmap," *ACM Comput. Surveys*, vol. 54, no. 11s, pp. 1–36, Jan. 2022.
- [59] N. Coppik, O. Schwahn, and N. Suri, "MemFuzz: Using memory accesses to guide fuzzing," in *Proc. 12th IEEE Conf. Softw. Test., Validation Verification (ICST)*, Apr. 2019, pp. 48–58.
- [60] C. Wen et al., "MEMLOCK: Memory usage guided fuzzing," in *Proc. IEEE/ACM 42nd Int. Conf. Softw. Eng. (ICSE)*, Oct. 2020, pp. 765–777.
- [61] T. Petsios, J. Zhao, A. D. Keromytis, and S. Jana, "SlowFuzz: Automated domain-independent detection of algorithmic complexity vulnerabilities," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Oct. 2017, pp. 2155–2168.
- [62] C. Lemieux, R. Padhye, K. Sen, and D. Song, "PerfFuzz: Automatically generating pathological inputs," in *Proc. 27th ACM SIGSOFT Int. Symp. Softw. Test. Anal.*, Jul. 2018, pp. 254–265.
- [63] Y. Shen, Y. Jiang, C. Xu, P. Yu, X. Ma, and J. Lu, "ReScue: Crafting regular expression DoS attacks," in *Proc. 33rd IEEE/ACM Int. Conf. Automated Softw. Eng. (ASE)*, Sep. 2018, pp. 225–235.
- [64] C. Aschermann, S. Schumilo, A. Abbasi, and T. Holz, "Ijon: Exploring deep state spaces via fuzzing," in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2020, pp. 1597–1612.
- [65] C. Lemieux and K. Sen, "FairFuzz: A targeted mutation strategy for increasing greybox fuzz testing coverage," in *Proc. 33rd IEEE/ACM Int. Conf. Automated Softw. Eng. (ASE)*, Sep. 2018, pp. 475–485.
- [66] M. Böhme, V.-T. Pham, M.-D. Nguyen, and A. Roychoudhury, "Directed greybox fuzzing," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Oct. 2017, pp. 2329–2344.
- [67] H. Chen et al., "Hawkeye: Towards a desired directed grey-box fuzzer," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Oct. 2018, pp. 2095–2108.
- [68] X. Zhu and M. Böhme, "Regression greybox fuzzing," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Nov. 2021, pp. 2169–2182.
- [69] C. E. Shannon, "A mathematical theory of communication," *Bell Syst. Tech. J.*, vol. 27, no. 3, pp. 379–423, Jul. 1948.
- [70] D. She, A. Shah, and S. Jana, "Effective seed scheduling for fuzzing with graph centrality analysis," in *Proc. IEEE Symp. Secur. Privacy (SP)*, May 2022, pp. 2194–2211.